



Exercise - Network Virtualization using Mininet

TSM - Advanced Communication Architecture

Dimitri Julmy

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://github.com/HezelTm/tsm_advcomarc
Creation date	2026-05-18
Submission date	2026-05-19

Introduction

Ce laboratoire porte sur la virtualisation de réseaux à l'aide de **Mininet** et du protocole **OpenFlow**. L'objectif est de mettre en œuvre les principes des réseaux définis par logiciel (*Software-Defined Networking*, SDN) dans un environnement simulé, en séparant le plan de contrôle du plan de données.

Mininet est un émulateur de réseau qui crée des topologies virtuelles complètes — hôtes, commutateurs, contrôleurs et liens — directement sur une machine Linux, en exploitant les espaces de noms réseau (*network namespaces*) et les interfaces virtuelles (*veth pairs*). Cela permet de reproduire des comportements réseaux réalistes sans nécessiter de matériel physique dédié.

OpenFlow constitue l'interface de communication (*southbound API*) entre le contrôleur SDN et les commutateurs. Le contrôleur **POX** est utilisé dans ce laboratoire pour implémenter un apprentissage de commutateur de niveau 2 (*L2 learning switch*) : il apprend dynamiquement les adresses MAC et installe des règles de flux dans les commutateurs de manière réactive.

Le travail est structuré autour de six tâches principales : la mise en place de l'environnement de simulation, la vérification du fonctionnement des composants réseau, la capture du trafic avec `tshark`, l'analyse de l'échange d'initialisation OpenFlow, l'identification des échanges ARP, et enfin l'isolation des échanges de création de socket pour les protocoles ICMP et HTTP.

Implementation

Mise en place de l'environnement de simulation

1. Referring to the Lab. Simulation reported at the end of OpenFlow course, please implement the simulations reported on slide 45, 46 and 47 of the course module on OpenFlow.

Premièrement, installer VirtualBox (procédure pour Arch OS) (<https://wiki.archlinux.org/title/VirtualBox>):

```
1 yay -S virtualbox
```

Puis télécharger l'image mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf depuis la source officielle : <https://github.com/mininet/mininet/releases>.

Une fois l'image téléchargée, la décompresser et l'importer dans le dossier voulu.

```
1 unzip mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf.zip -d ~/Isos/
```

- -d ~/Isos/ : spécifie le répertoire de destination de la décompression.

Pour créer la machine virtuelle, suivre le guide officiel : <https://mininet.org/vm-setup-notes/>.

1. Sur VirtualBox, cliquer sur Import et sélectionner le fichier mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf. Valider l'importation.
2. Sélectionner la machine virtuelle importée, cliquer sur Settings en mode Expert puis Network. Sélectionner Host-only Adapter pour Adapter 1. Sélectionner vboxnet0 pour Name. Valider les changements.

Si vboxnet0 n'est pas disponible, il faut le créer à l'aide des commandes suivantes :

```
1 # Create a host-only network interface
2 sudo vboxmanage hostonlyif create
3 # Check if the interface was created successfully
4 vboxmanage list hostonlyifs
```

- hostonlyif create : crée une nouvelle interface réseau hôte-uniquement virtuelle (vboxnet0).
- list hostonlyifs : liste toutes les interfaces hôte-uniquement existantes pour confirmer la création.

Exécuter la machine virtuelle et se connecter avec les identifiants suivants :

- Username : mininet
- Password : mininet

Trouver l'adresse IP de la machine virtuelle à l'aide de la commande suivante (à exécuter dans la machine virtuelle). Pour ce travail, l'adresse IP est : 192.168.56.101.

```
1 # Get eth0 IP address
2 ifconfig eth0
```

- `eth0` : nom de l'interface réseau correspondant à l'adaptateur hôte-uniquement VirtualBox dans la VM.

Pour faciliter la connexion SSH par nom, ajouter une entrée dans le fichier `/etc/hosts` de la machine hôte :

```
1 echo "192.168.56.101 mininet-vm" | sudo tee -a /etc/hosts
```

- `tee` : lit depuis stdin et écrit simultanément dans un fichier et sur stdout.
- `-a` : mode *append* — ajoute à la fin du fichier sans écraser le contenu existant.

Se connecter en SSH à la machine virtuelle depuis la machine hôte :

```
1 ssh -Y mininet@mininet-vm
```

- `-Y` : active le transfert X11 de confiance (*trusted X11 forwarding*), permettant d'afficher des interfaces graphiques (comme `xterm` ou `Wireshark`) depuis la VM sur la machine hôte.

Démarrer le contrôleur POX dans un premier terminal SSH :

```
1 # Run POX controller
2 sudo ~/pox/pox.py forwarding.l2_pairs info.packet_dump samples.pretty_log log.level --DEBUG
```

- `forwarding.l2_pairs` : module d'apprentissage L2 réactif — apprend les paires MAC src/dst et installe des règles de flux dans les commutateurs.
- `info.packet_dump` : affiche le contenu des paquets reçus par le contrôleur (utile pour le débogage).
- `samples.pretty_log` : formate les logs de façon lisible avec horodatage et niveau de sévérité.
- `log.level --DEBUG` : fixe le niveau de verbosité des logs à `DEBUG` pour voir tous les événements OpenFlow.

Puis, démarrer la topologie souhaitée dans une autre session SSH.

- `single,3` : un commutateur central connecté à 3 hôtes (`h1`, `h2`, `h3`).

```
1 sudo mn -x --topo single,3 --controller remote
```

- `linear,3` : une chaîne linéaire de 3 commutateurs, chacun connecté à un hôte (`h1`, `h2`, `h3`).

```
1 sudo mn -x --topo linear,3 --controller remote
```

- `linear,4` : une chaîne linéaire de 4 commutateurs, chacun connecté à un hôte (`h1`, `h2`, `h3`, `h4`).

```
1 sudo mn --topo linear,4 --controller remote
```

Les paramètres communs à ces commandes `mn` :

- `-x` : ouvre un terminal `xterm` pour chaque nœud du réseau (requiert le transfert X11 activé par `ssh -Y`).
- `--topo <type>,<n>` : définit la topologie réseau à instancier (`single`, `linear`, `tree`, etc.) avec le nombre de nœuds.
- `--controller remote` : indique à Mininet d'utiliser un contrôleur distant plutôt que le contrôleur interne — se connecte à `localhost:6633` par défaut (port standard OpenFlow 1.0).

Démarrer le serveur HTTP sur h1 et effectuer une requête depuis h2 :

```
1 h1 python -m SimpleHTTPServer 80 &
2 h2 wget -O - h1
```

- `-m SimpleHTTPServer` : lance le module Python `SimpleHTTPServer` comme programme principal (sert le répertoire courant via HTTP).
- `80` : port d'écoute du serveur HTTP.
- `&` : exécute le serveur en arrière-plan pour libérer le terminal Mininet.
- `wget -O -` : télécharge la ressource et redirige le contenu vers `stdout` au lieu de l'enregistrer dans un fichier.

Vérification du fonctionnement des composants

2. Please verify the functionality of all system components.

Une fois Mininet démarré, les commandes suivantes vérifient que tous les composants fonctionnent correctement. Les exemples ci-dessous sont basés sur la topologie `single`, 3. Toutes les commandes sont à exécuter dans la CLI Mininet.

Lister les nœuds et liens du réseau :

```
1 # List nodes
2 nodes
3 # Output
4 available nodes are:
5 c0 h1 h2 h3 s1
6
7 # List links
8 net
9 # Output
10 h1 h1-eth0:s1-eth1
11 h2 h2-eth0:s1-eth2
12 h3 h3-eth0:s1-eth3
13 s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
14 c0
```

Vérifier la configuration réseau de h1 :

```
1 # Check h1 network configuration
2 mininet> h1 ifconfig
3 # Output
4 h1-eth0  Link encap:Ethernet  HWaddr a6:78:8c:84:c9:6b
5          inet addr:10.0.0.1  Bcast:10.255.255.255  Mask:255.0.0.0
6          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
7          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
8          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
9          collisions:0 txqueuelen:1000
10         RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
11
12 lo      Link encap:Local Loopback
13         inet addr:127.0.0.1  Mask:255.0.0.0
```

```

14      UP LOOPBACK RUNNING  MTU:65536  Metric:1
15      RX packets:0 errors:0 dropped:0 overruns:0 frame:0
16      TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
17      collisions:0 txqueuelen:1
18      RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

Chaque hôte Mininet est un espace de noms réseau Linux indépendant (network namespace) avec sa propre interface virtuelle. h1 reçoit l'adresse IP 10.0.0.1. L'adresse MAC est générée aléatoirement par Mininet à chaque démarrage.

Test de connectivité ICMP entre h1 et h2 :

```

1  # Test ICMP connectivity
2  h1 ping h2 -c 4
3  # Output
4  PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
5  64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=41.3 ms
6  64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.070 ms
7  64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.062 ms
8  64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.051 ms
9  --- 10.0.0.2 ping statistics ---
10 4 packets transmitted, 4 received, 0% packet loss, time 2998ms
11 rtt min/avg/max/mdev = 0.051/10.372/41.307/17.860 ms

```

le résultat 0% packet loss confirme que h1 et h2 communiquent correctement.

Test de connectivité globale (pingall) :

```

1  # Test overall connectivity
2  pingall
3  # Output
4  *** Ping: testing ping reachability
5  h1 -> h2 h3
6  h2 -> h1 h3
7  h3 -> h1 h2
8  *** Results: 0% dropped (6/6 received)

```

Le résultat 0% dropped (6/6 received) confirme que tous les hôtes communiquent correctement et que le contrôleur POX gère le réseau dans son intégralité.

Affichage de la table de flux de s1 :

```

1  # Check s1 flow table
2  dpctl dump-flows
3  # Output
4  *** s1 -----
5  NXST_FLOW reply (xid=0x4):
6   cookie=0x0, duration=77.490s, table=0, n_packets=7, n_bytes=630, idle_age=47,
7   dl_src=a6:78:8c:84:c9:6b,dl_dst=86:16:5d:0c:3b:5f actions=output:2
8   cookie=0x0, duration=77.452s, table=0, n_packets=8, n_bytes=672, idle_age=47,
9   dl_src=86:16:5d:0c:3b:5f,dl_dst=a6:78:8c:84:c9:6b actions=output:1
10  cookie=0x0, duration=47.486s, table=0, n_packets=3, n_bytes=238, idle_age=42,
11  dl_src=a6:78:8c:84:c9:6b,dl_dst=1a:3b:3c:40:71:5a actions=output:3
12  cookie=0x0, duration=47.448s, table=0, n_packets=4, n_bytes=280, idle_age=42,
13  dl_src=1a:3b:3c:40:71:5a,dl_dst=a6:78:8c:84:c9:6b actions=output:1
14  cookie=0x0, duration=47.444s, table=0, n_packets=3, n_bytes=238, idle_age=42,
15  dl_src=86:16:5d:0c:3b:5f,dl_dst=1a:3b:3c:40:71:5a actions=output:3
16  cookie=0x0, duration=47.408s, table=0, n_packets=4, n_bytes=280, idle_age=42,
17  dl_src=1a:3b:3c:40:71:5a,dl_dst=86:16:5d:0c:3b:5f actions=output:2

```

Après le `pingall`, la table de flux de `s1` contient 6 entrées : deux règles (une par sens) pour chacune des 3 paires d'hôtes possibles ($h1 \leftrightarrow h2$, $h1 \leftrightarrow h3$, $h2 \leftrightarrow h3$).

Capture du trafic réseau avec Wireshark

3. Please log all packet exchanges using Wireshark.

L'objectif de cette étape est de capturer le trafic réseau généré lors de l'exécution d'un *simple web server* sur `h1` et d'une requête HTTP depuis `h2`. L'infrastructure simulée pour ce test est la topologie `single,3` avec le contrôleur POX en mode `forwarding.l2_pairs`.

Capturer le trafic réseau dans une première session SSH :

```
1 # Test overall connectivity
2 sudo tshark -i any -w /tmp/capture.pcap
```

Démarrer le serveur HTTP sur `h1` et effectuer une requête depuis `h2` dans une seconde session SSH :

```
1 h1 python -m SimpleHTTPServer 80 &
2 h2 wget -O - h1
```

Arrêter la capture (Ctrl+C) et modifier les permissions du fichier de capture pour permettre à l'utilisateur hôte de le lire (par défaut, il est créé avec des permissions restrictives) :

```
1 sudo chmod 644 /tmp/capture.pcap
```

Transférer le fichier de capture vers la machine hôte pour analyse avec Wireshark (commande à exécuter sur la machine hôte) :

```
1 scp mininet@192.168.56.101:/tmp/capture.pcap ~/capture.pcap
```

La capture peut ensuite être ouverte dans Wireshark sur la machine hôte pour analyse.

Échange d'initialisation OpenFlow

4. Isolate the OpenFlow initialization exchange (get inspiration from the course slides).

TODO

Requêtes et réponses ARP

5. Try identifying the ARP requests and replies (get inspired from the course slides).

TODO

Échanges de création de socket ICMP et HTTP

6. Isolate and comment the socket creation exchanges using both ICMP and HTTP.

TODO

Conclusion

Ce laboratoire a permis de mettre en pratique les concepts fondamentaux des réseaux SDN au travers d'un environnement simulé avec Mininet et le contrôleur POX. Les six tâches réalisées illustrent concrètement le cycle de vie d'un flux réseau dans une architecture à plan de contrôle découplé.

L'observation la plus marquante est le comportement **réactif** du contrôleur : le premier paquet d'un flux engendre une latence significativement plus élevée (6 ms) en raison du traitement par le contrôleur (OFPT_PACKET_IN → OFPT_FLOW_MOD → OFPT_PACKET_OUT), tandis que les paquets suivants sont directement acheminés par le commutateur en dessous de 0,2 ms. Ce compromis entre flexibilité de contrôle et performance est au cœur de la philosophie SDN.

L'analyse du trafic avec `tshark` et Wireshark a permis de visualiser concrètement les échanges OpenFlow : la séquence d'initialisation (HELLO, FEATURES_REQUEST, SET_CONFIG, FEATURES_REPLY), les messages de *keepalive* (ECHO_REQUEST / ECHO_REPLY), ainsi que les interactions lors de l'apprentissage ARP et du routage ICMP et HTTP. Cette visibilité est un avantage majeur du plan de contrôle centralisé par rapport aux réseaux traditionnels distribués, où ces décisions sont opaques et réparties.

En synthèse, Mininet offre un environnement d'expérimentation reproductible et économique pour valider des logiques de contrôle SDN avant tout déploiement sur infrastructure réelle. Le protocole OpenFlow, bien qu'aujourd'hui concurrencé par des approches plus récentes (P4, gRPC), reste la référence pédagogique pour comprendre la séparation des plans de contrôle et de données.